

a. ¿Por qué las peticiones HTTP devuelven un Observable y no la data directa?

Porque las peticiones son asincronicas, el resultado llega en el futuro, un Observable es un flujo que emite valores cuando están disponibles, permite cancelarlos y encadenar operadores como map o catchError, si devolviera la data directa el código tendría que bloquearse esperando la respuesta.

b. ¿Qué diferencia hay frente a una Promise?

Una Promise solo emite un valor y no se puede cancelar. Un Observable puede emitir múltiples valores, es cancelable con unsubscribe y permite operadores reactivos de RxJS. Para HTTP un solo valor basta.

c. ¿Qué ocurre si nunca se llama .subscribe()?

La petición HTTP no se ejecuta en angular pues los observables de HttpClientson lazy solo hacen la petición real cuando alguien se suscribe

d. ¿Qué ventaja tiene inyectar inject(CountryService) frente a instanciar con new CountryService()?

Con la inyección Angular maneja el ciclo de vida y garantiza una sola instancia compartida con new se crea una instancia aislada sin acceso a las dependencias propias del servicio lo que rompe la cadena de inyección.

e. ¿Cómo facilita la inyección de dependencias las pruebas unitarias?

Permite reemplazar el servicio real por un mock en los tests, sin cambiar el componente. Basta con proveer una implementación falsa en el TestBed.

f. ¿Por qué centralizar el manejo de errores en un interceptor?

Evita duplicar bloques try/catch o catchError en cada component, un solo interceptor captura todos los errores HTTP de la aplicación, mantiene el código DRY y facilita cambiar la estrategia de manejo de errores en un solo lugar

g. Menciona dos casos de uso adicionales para un interceptor:

- Agregar el token de autenticación a cada petición saliente.
- Mostrar un spinner global de carga mientras hay peticiones en vuelo.

h. ¿Por qué se usa @Input() para pasar la ciudad a CityDetailComponent?

Porque la ciudad ya está disponible en el componente padre es decir el citylistcomponent, hacer otro GET a /api/cities/{id} sería una petición innecesaria, el patrón Maestro-Detalle pasa el objeto completo por @Input(), esto siendo mucho mas eficiente.

i. ¿Qué ventaja tiene que country venga como objeto anidado y no solo el id?

El frontend no necesita hacer una segunda petición para obtener el nombre del país ya que al tener city.country.name disponible se reduce la cantidad de llamadas al backend y el código del frontend es más simple.